Part 2：OpenClaw 记忆与配置修改

> *给 OpenClaw 接上"大脑"、带它认识你、让它开口说第一句话——完成这三步，你就拥有了一个真正能干活的 AI 助手。*

引言

上一章结束时，我们在终端上看到了 OpenClaw 的版本号和 `doctor` 诊断报告。安装是成功了，但说句实话——它现在还什么都干不了。

原因很简单：**OpenClaw 本身并不包含一个大模型。**它是一个"助手操作系统"，是指挥官、调度员、工具箱——但它没有自带"大脑"。就像你刚组装好一台电脑，硬件齐全了，但还没装操作系统、没连上网，打开电源只有一个光标在那闪。

这一章，我们要做三件事：

1. **给它接上大脑** —— 注册大模型 API，让 OpenClaw 有"思考能力"
2. **完成首次配置** —— 运行 `openclaw setup`，初始化所有配置文件
3. **让它开口说话** —— 启动 Gateway，打开 Dashboard，进行第一次对话

完成之后，你的 OpenClaw 不仅会说话、会思考，你还可以给它改名字、改性格——甚至让它用东北话和你聊天。

2.1 什么是模型调用？大模型 API 介绍

为什么 OpenClaw 需要大模型 API？

在上手操作之前，有三个概念必须先搞清楚。它们不难，但会贯穿后面所有章节。

先回答最基本的问题：**OpenClaw 为什么不能像 ChatGPT 一样直接聊天？**

因为 OpenClaw 的定位不是"聊天机器人"，而是"助手操作系统"。它管理技能、记忆、消息通道、定时任务……但它自己并不"思考"。思考的工作，交给外部的大模型来做。

用一个类比来理解：

> ***OpenClaw 是手机，大模型是 SIM 卡。** 手机再好——屏幕多大、芯片多快——不插 SIM 卡就打不了电话。OpenClaw 再强大，不接入大模型 API 就无法回答你的问题。*



API 是什么？

API 的全称是 **Application Programming Interface**（应用程序编程接口）。名字很唬人，但概念很简单：

> ***API 就像餐厅的菜单。** 你（用户）通过菜单（API）向厨房（大模型）点菜（发请求），厨房做好菜（计算结果）端给你。你不需要会做饭，只需要会看菜单、会点菜。*

OpenClaw 就是帮你"点菜"的服务员——它把你的请求翻译成 API 能理解的格式，发给大模型，再把结果翻译成你能读懂的回答。

API Key 是什么？

API Key 是你调用 API 时的**身份凭证**。

> *类比：会员卡。去餐厅吃饭要先办会员卡，卡号就是你的 API Key。每次下单时出示会员卡，系统验证你的身份、从你的余额中扣费。*

API Key 的格式通常是一串字母和数字的组合，比如 `sk-abc123def456...`。拿到之后要像密码一样保管——**不要截图发到群里、不要提交到 GitHub、不要写在任何公开的地方。**

Token 是什么？

大模型 API 不按"次数"收费，而是按 **Token** 数量收费。

Token 是文本的最小计量单位。大致换算：

- 1 个中文字 ≈ 1-2 个 tokens
- 1 个英文单词 ≈ 1 个 token
- 100 个 tokens ≈ 75 个英文单词 ≈ 50 个中文字

> *类比：打电话按分钟计费。 你发给大模型的消息（输入）和它返回的回答（输出）都要消耗 token。输入和输出的单价不同，通常输出比输入贵。*

用一张表总结：

概念	一句话解释	类比
API	调用大模型的标准接口	餐厅菜单
API Key	你的身份凭证	会员卡号
Token	文本的最小计量单位	电话按分钟计费

搞清楚这三个概念，接下来的操作就不会迷路了。

2.2 大模型 API 注册与接入：三大平台实操

三大推荐平台概览

OpenClaw 支持几乎所有兼容 OpenAI 协议的大模型 API。面对琳琅满目的选项，我给你推荐三个最适合入门的平台：

平台	推荐度	免费额度	是否需翻墙	推荐模型	适合人群
DeepSeek	⭐⭐⭐ 首选	注册送 500 万 tokens	不需要	deepseek-chat (V3.2)	所有人
Kimi Code	⭐⭐ 备选	有免费额度	不需要	moonshot-v1-8k	国内用户
OpenAI	⭐ 进阶	无	需要翻墙	gpt-4o	有梯子的用户

我的建议：直接选 DeepSeek。 原因很简单——免费额度最多（500 万 tokens 足够你用很久）、国内直连速度快、模型能力强劲。后面的操作我们也以 DeepSeek 为主来演示。

方案一：DeepSeek API 注册（推荐首选）

DeepSeek 是深度求索公司推出的大模型 API 服务。它的 V3.2 模型在多个基准测试中表现优异，而价格只有 GPT-4o 的十分之一。更重要的是：**注册就送 500 万免费 tokens，不需要绑信用卡。**

注册步骤：

1. 访问 DeepSeek 开放平台：`platform.deepseek.com`
2. 点击"注册"，用手机号完成注册
3. 注册成功后，自动获得 **500 万免费 tokens**
4. 进入 Dashboard → 左侧菜单 **API Keys** → 点击 **创建新 Key**
5. 给你的 Key 起个名字（比如"OpenClaw"），点击创建

> *界面截图：DeepSeek API Keys 管理页面*
>
> 
>

> ⚠️ *重要提醒：API Key 创建后只显示一次！点击"复制"按钮把它保存到安全的地方（比如备忘录或密码管理器）。关闭页面后就再也看不到完整的 Key 了——只能删除重建。*

DeepSeek 定价速览：

模型	缓存命中输入	新输入	输出	上下文窗口
deepseek-chat (V3.2)	$0.07/M	$0.27/M	$1.10/M	128K
deepseek-reasoner (V3.2)	$0.14/M	$0.55/M	$2.19/M	128K

> *M = 百万 tokens。按最贵的输出价格算：500 万免费 tokens ≈ $5.5 的输出，日常使用可以撑很久。*

方案二：Kimi Code（备选方案）

Kimi Code 是 Moonshot（月之暗面）推出的编程 AI 服务，与 OpenClaw 兼容。

注册步骤：

1. 访问 Kimi Code 官网，用手机号注册
2. 在设置中获取 API Key
3. 记录 Base URL 和 Key 格式

Kimi Code 的优势是国内直连速度快，且对中文的理解非常好。如果 DeepSeek 遇到服务不稳定的情况，Kimi Code 是一个不错的备用选择。

方案三：OpenAI API（进阶可选）

OpenAI 是大模型领域的"鼻祖"，GPT-4o、GPT-5.4 都是业界顶级模型。但对于国内用户来说有两个门槛：

1. **需要翻墙**才能访问 `platform.openai.com`
2. **需要绑定信用卡**才能使用 API

如果你有海外支付手段并且已经在使用梯子，OpenAI 仍然是最成熟、生态最完善的选择。注册步骤：访问 `platform.openai.com` → 注册账号 → API Keys → 创建新 Key。

补充说明：阿里百炼 Coding Plan

你可能在一些教程中看到过推荐"阿里百炼 Coding Plan"——这是阿里云专门为编程工具优化的 API 套餐，价格很低（Lite 仅 ¥40/月）且支持多个顶级模型。但截至目前（2026 年 3 月），**Coding Plan 已停止申请**（名额已满），暂时无法新注册。如果你已有 Coding Plan 账号，Key 格式为 `sk-sp-xxxxx`（注意：与百炼通用 Key 不通用）。

2.3 首次配置：初始化 OpenClaw 环境

拿到 API Key 之后，我们需要告诉 OpenClaw 两件事：

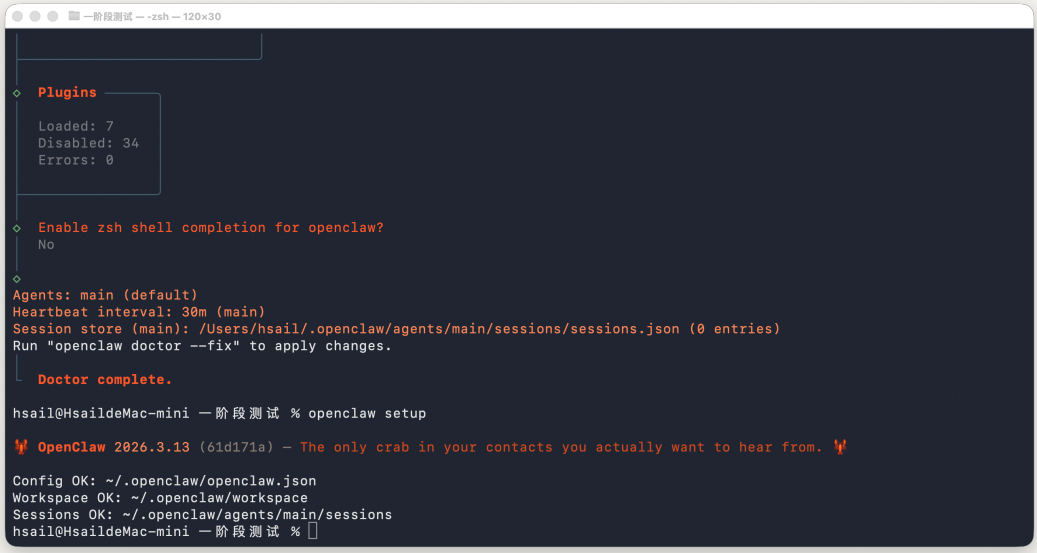
1. 去哪里调用大模型（Base URL）
2. 用什么身份调用（API Key）

第一步：初始化配置目录

OpenClaw 有一个专门的初始化命令 `openclaw setup`，它会自动创建所有必要的配置文件和目录。

在终端中执行：

```
$ openclaw setup
Wrote ~/.openclaw/openclaw.json
Workspace OK: ~/.openclaw/workspace
Sessions OK: ~/.openclaw/agents/main/sessions
```

> 📸 *终端截图：运行 openclaw setup*
>
> 
>

这条命令做了三件事：

- 创建了主配置文件 `~/.openclaw/openclaw.json`
- 初始化了工作空间目录 `~/.openclaw/workspace/`（包含 SOUL.md、USER.md 等特殊文件）
- 创建了会话存储目录 `~/.openclaw/agents/main/sessions/`

> 💡 *关于 **`openclaw onboard`**：OpenClaw 还有一个交互式向导命令 `openclaw onboard`，会一步步引导你选择模型、输入 API Key、配置 Gateway。但在课程中，我们选择手动配置的方式——这样你不仅能完成配置，还能理解每一步背后的原理。*

第二步：配置 API Key（.env 文件）

OpenClaw 把敏感信息（如 API Key）存放在一个单独的 `.env` 环境变量文件中，和主配置文件分离。这是一个好的安全实践——配置文件可以公开讨论，但 `.env` 文件永远不应该泄露。

用文本编辑器创建 `~/.openclaw/.env` 文件，写入你的 DeepSeek API Key：

```
# 用 nano 编辑器创建 .env 文件
$ nano ~/.openclaw/.env
```

写入以下内容（把 `sk-your-deepseek-key` 替换成你自己的 Key）：

```
DEEPSEEK_API_KEY=sk-your-deepseek-key
```

保存退出（nano 中按 `Ctrl + X`，然后按 `Y` 确认，回车保存）。

验证文件内容：

```
$ cat ~/.openclaw/.env
DEEPSEEK_API_KEY=sk-your-deepseek-key
```

> ⚠️ *安全提醒：`.env` 文件中存储的是你的 API Key，相当于密码。不要发给别人，不要上传到任何公开平台。*

第三步：配置模型提供商（openclaw.json）

接下来要告诉 OpenClaw："我要用 DeepSeek 作为大模型提供商。"

我们使用 OpenClaw 自带的 `config set` 命令来修改配置。这比直接编辑 JSON 文件更安全——OpenClaw 有严格的 JSON Schema 校验，手动编辑一不小心写错格式，Gateway 会拒绝启动。

配置模型合并模式：

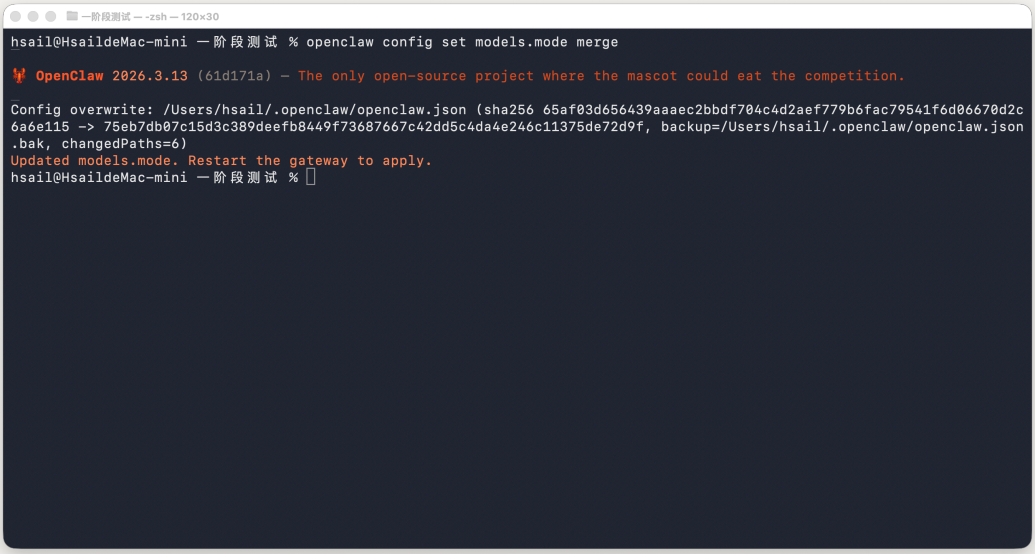
```
$ openclaw config set models.mode merge
Updated models.mode. Restart the gateway to apply.
```

> *`merge` 模式表示新的 provider 配置会合并到已有配置中，不会覆盖其他设置。*

配置 DeepSeek 提供商：

```
$ openclaw config set models.providers.deepseek '{
  "baseUrl": "https://api.deepseek.com/v1",
  "apiKey": "${DEEPSEEK_API_KEY}",
  "api": "openai-completions",
  "models": [{
    "id": "deepseek-chat",
    "name": "DeepSeek Chat (v3.2)",
    "reasoning": false,
```

```
    "input": ["text"],
    "cost": {"input": 2.8e-7, "output": 4.2e-7},
    "contextWindow": 128000,
    "maxTokens": 8192
  }]
}'
Updated models.providers.deepseek. Restart the gateway to
apply.
```

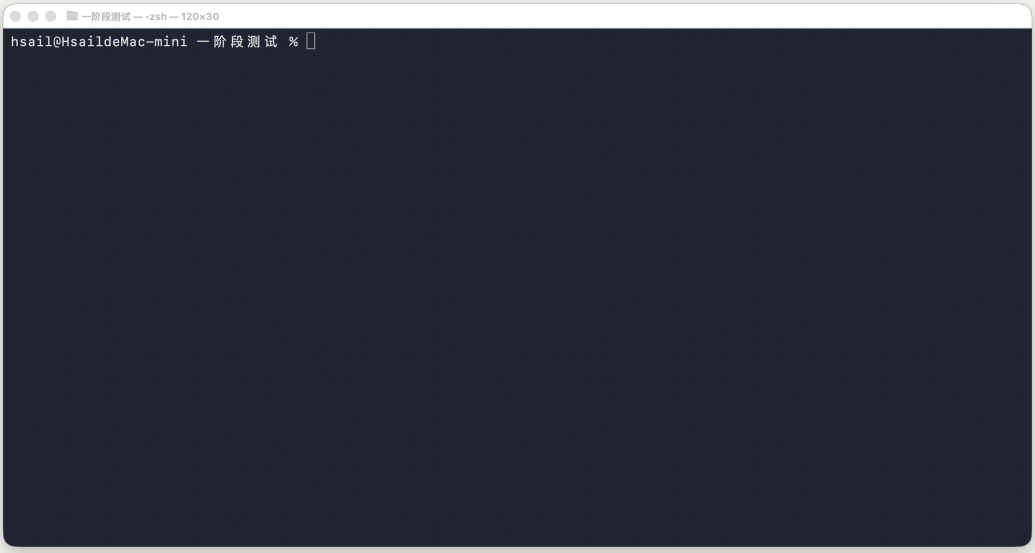
> 📸 *终端截图：配置 DeepSeek provider*
>
> 
>

注意看 `"apiKey": "${DEEPSEEK_API_KEY}"`——这里用的是 `${...}` 语法，表示从 `.env` 文件中读取环境变量。这样 API Key 就不会直接暴露在配置文件里。

设置默认模型：

```
$ openclaw config set agents.defaults.model.primary
"deepseek/deepseek-chat"
Updated agents.defaults.model.primary. Restart the gateway to
apply.
```

这告诉 OpenClaw："默认使用 DeepSeek 的 deepseek-chat 模型来回答问题。"

> 📸 *终端截图：设置默认模型*
>
> 
>

第四步：验证配置文件

让我们看一下最终的配置文件长什么样：

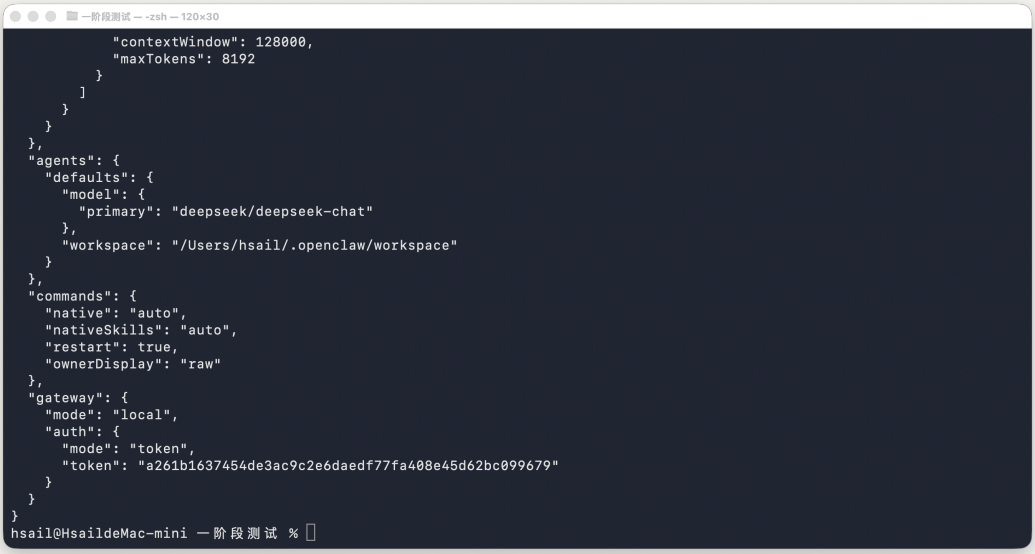
```
$ cat ~/.openclaw/openclaw.json
```

输出（已格式化）：

```
{
  "meta": {
    "lastTouchedVersion": "2026.3.13",
    "lastTouchedAt": "2026-03-18T08:34:27.636Z"
  },
  "models": {
    "mode": "merge",
    "providers": {
      "deepseek": {
        "baseUrl": "https://api.deepseek.com/v1",
        "apiKey": "${DEEPSEEK_API_KEY}",
        "api": "openai-completions",
        "models": [
          {
            "id": "deepseek-chat",
            "name": "DeepSeek Chat (v3.2)",
            "reasoning": false,
```

```
            "input": ["text"],
            "cost": { "input": 2.8e-7, "output": 4.2e-7 },
            "contextWindow": 128000,
            "maxTokens": 8192
          }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": {
        "primary": "deepseek/deepseek-chat"
      },
      "workspace": "/Users/你的用户名/.openclaw/workspace"
    }
  },
  "gateway": {
    "mode": "local"
  }
}
```

📸 *终端截图：查看完整配置文件*



用一张表来对照每个字段的含义：

字段	含义
`meta`	元数据，记录最后修改的版本和时间
`models.mode`	`merge` = 合并配置，不覆盖已有设置
`models.providers.deepseek`	DeepSeek API 的连接信息
`models.providers.deepseek.apiKey`	通过 `${DEEPSEEK_API_KEY}` 引用 .env 中的密钥
`agents.defaults.model.primary`	默认使用的模型（deepseek/deepseek-chat）
`gateway.mode`	`local` = 本地模式，Gateway 运行在你的 Mac 上

如果你以后想添加其他模型提供商（比如 Kimi Code 或 OpenAI），只需要在 `providers` 下面加一个新的配置块，再在 `.env` 中加上对应的 API Key 即可。`merge` 模式会自动合并，互不影响。

2.4 启动 Gateway 与打开 Dashboard

Gateway 是什么？

Gateway 是 OpenClaw 的**核心服务**——所有功能都通过它提供。

> *类比：**Gateway** 就像酒店的前台。所有客人（请求）都要先经过前台（Gateway），由前台分配房间（调度 Agent）、安排服务（调用技能）、管理入住退房（会话管理）。没有前台，整个酒店就运转不起来。*

Gateway 的技术细节：

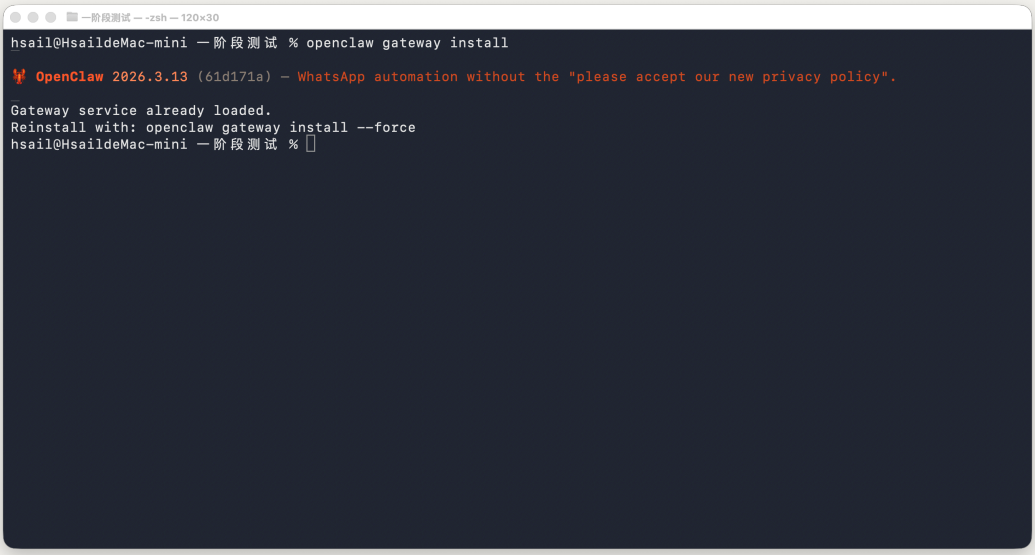
- 基于 **WebSocket** 协议，支持实时双向通信
- 默认端口 **18789**
- 默认绑定 **127.0.0.1**（只能从你自己的电脑访问，安全第一）
- 支持 Token 认证（自动生成）

实操：安装并启动 Gateway

第一步：安装 Gateway 为系统服务

```
$ openclaw gateway install
No gateway token found. Auto-generated one and saving to
config.

Installed LaunchAgent:
~/Library/LaunchAgents/ai.openclaw.gateway.plist
Logs: ~/.openclaw/logs/gateway.log
```

> 📸 *终端截图：安装 Gateway 服务*
>
> 
>

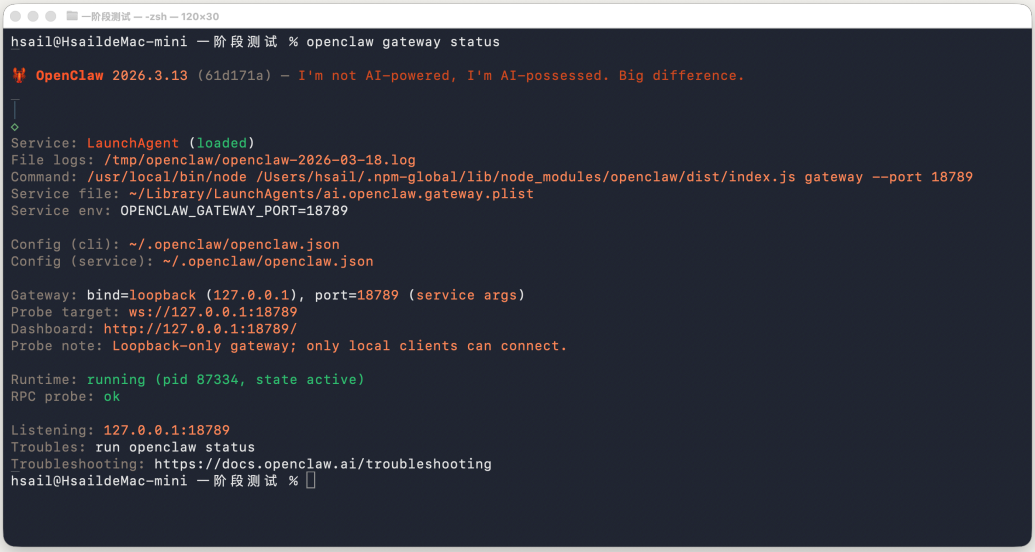
这条命令做了两件事：

1. 自动生成了认证 **Token** —— 保存到了配置文件的 `gateway.auth` 中，后续访问 Dashboard 会用到
2. 注册了 **macOS LaunchAgent** —— 这意味着 Gateway 会在你每次开机时自动启动，不需要手动运行

> 💡 *什么是 **LaunchAgent**？ 这是 macOS 的后台服务管理机制。注册之后，即使你重启电脑，Gateway 也会自动在后台启动——你的 AI 助手永远在线。*

第二步：启动 Gateway

```
$ openclaw gateway start
Restarted LaunchAgent: gui/501/ai.openclaw.gateway
```

> 📸 *终端截图：启动 Gateway*
>
> 
>

第三步：确认 Gateway 运行状态

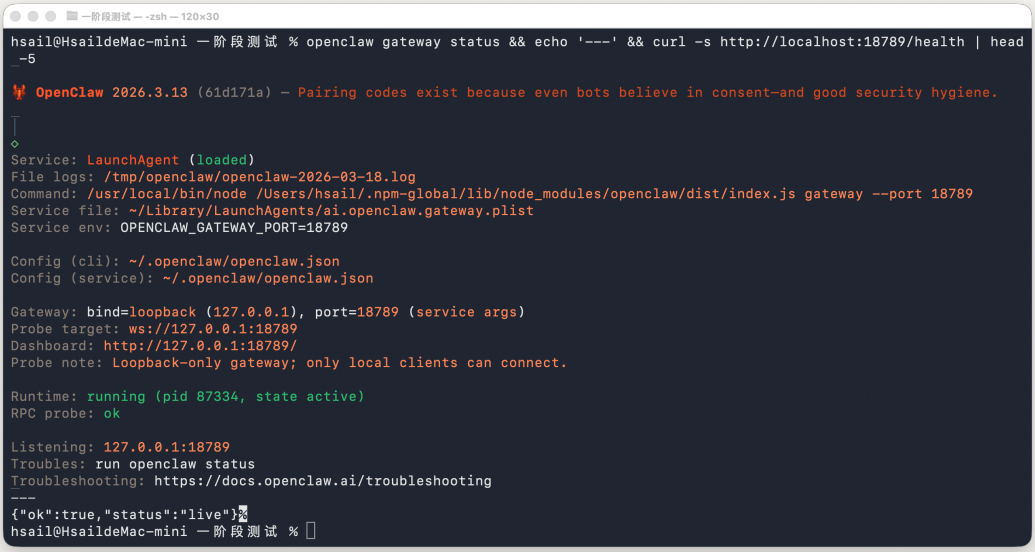
等待两三秒，让 Gateway 完全启动，然后检查状态：

```
$ openclaw gateway status
Service: LaunchAgent (loaded)
Command: /usr/local/bin/node ~/.npm-
global/lib/node_modules/openclaw/dist/index.js gateway --port
18789

Gateway: bind=loopback (127.0.0.1), port=18789 (service args)
Dashboard: http://127.0.0.1:18789/

Runtime: running (pid 77280, state active)
RPC probe: ok

Listening: 127.0.0.1:18789
```

> 📸 *终端截图：检查 Gateway 状态*
>
> 
>

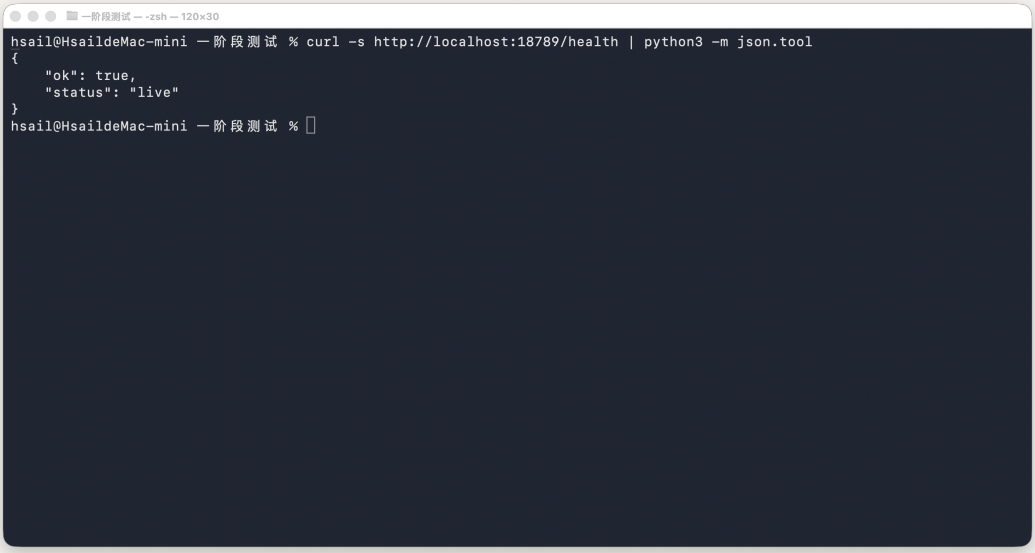
关键信息解读：

项	值	含义
Service	LaunchAgent (loaded)	系统服务已加载
Runtime	running (state active)	正在运行
RPC probe	ok	连接测试通过
Listening	127.0.0.1:18789	监听本机 18789 端口

看到 `Runtime: running` 和 `RPC probe: ok`，说明 Gateway 已经成功启动。

第四步：再来一次健康检查

```
$ openclaw gateway health
Gateway Health
OK (0ms)
```

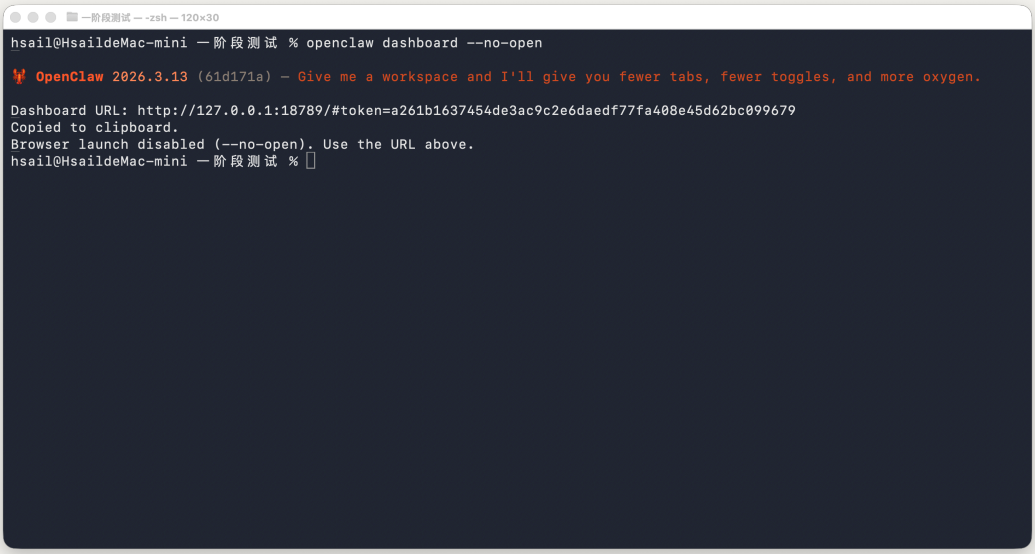
> 📸 *终端截图：Gateway 健康检查*
>
> 
>

`OK (0ms)` —— 完美，Gateway 健康运行，响应时间几乎为零（毕竟是本地连接）。

实操：打开 Dashboard

Dashboard 是 OpenClaw 的 Web 管理界面——你和 AI 对话、管理会话、查看技能、修改设置，都在这里完成。

```
$ openclaw dashboard --no-open
Dashboard URL: http://127.0.0.1:18789/#token=a261b163...
Copied to clipboard.
Browser launch disabled (--no-open). Use the URL above.
```

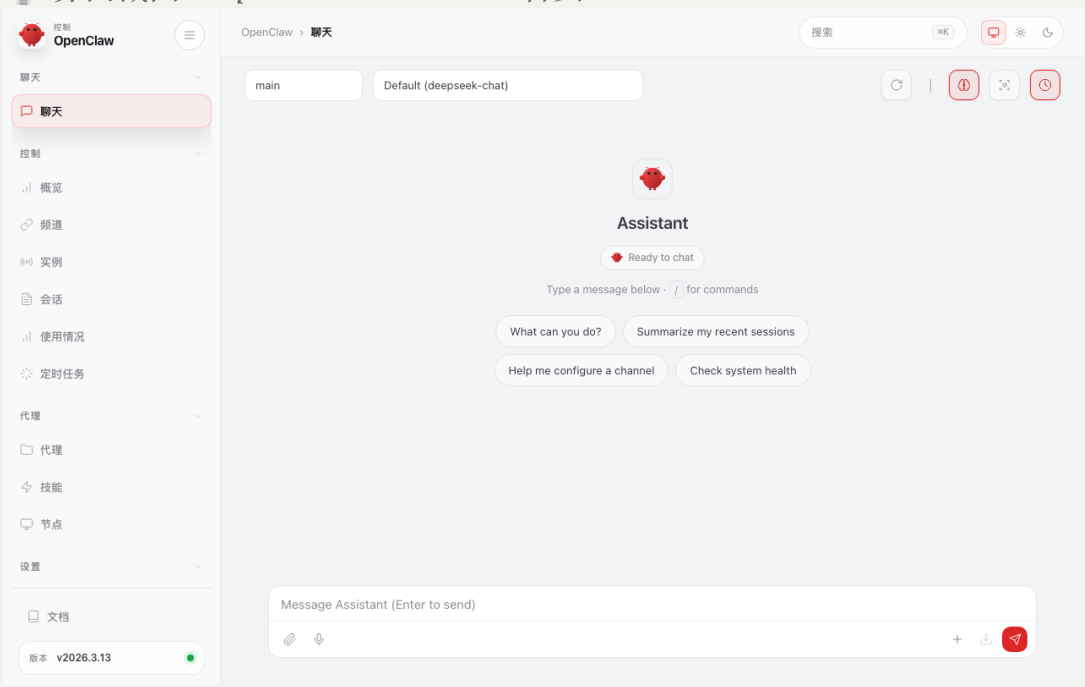
> 📸 *终端截图：获取 Dashboard URL*
>
> 
>

> 💡 `--no-open` 参数表示只输出 URL 但不自动打开浏览器。如果你想直接打开，用 `openclaw dashboard`（不加 `--no-open`），浏览器会自动跳转。

把这个 URL 复制到浏览器中打开，你就能看到 OpenClaw Dashboard 的界面了。

注意看 URL 中的 `#token=...`——这就是前面自动生成的认证 Token。Dashboard 通过 URL 中的 Token 自动完成认证，无需手动输入密码。

> 🖥️ *界面截图：OpenClaw Dashboard 首页*



Dashboard v2（2026.3.12+ 版本）的主要功能区：

功能区	说明
对话区域	中间最大的区域，输入消息和查看 AI 回复
会话列表	左侧栏，管理多个对话会话
技能面板	查看已启用的技能列表
设置面板	修改模型、查看配置、管理设备
命令面板	快捷命令入口（Command Palette）

2.5 首次对话：让 AI 助手开口说话

这一刻你等了两章了——终于可以和自己的 AI 助手说话了。

对话一：打个招呼

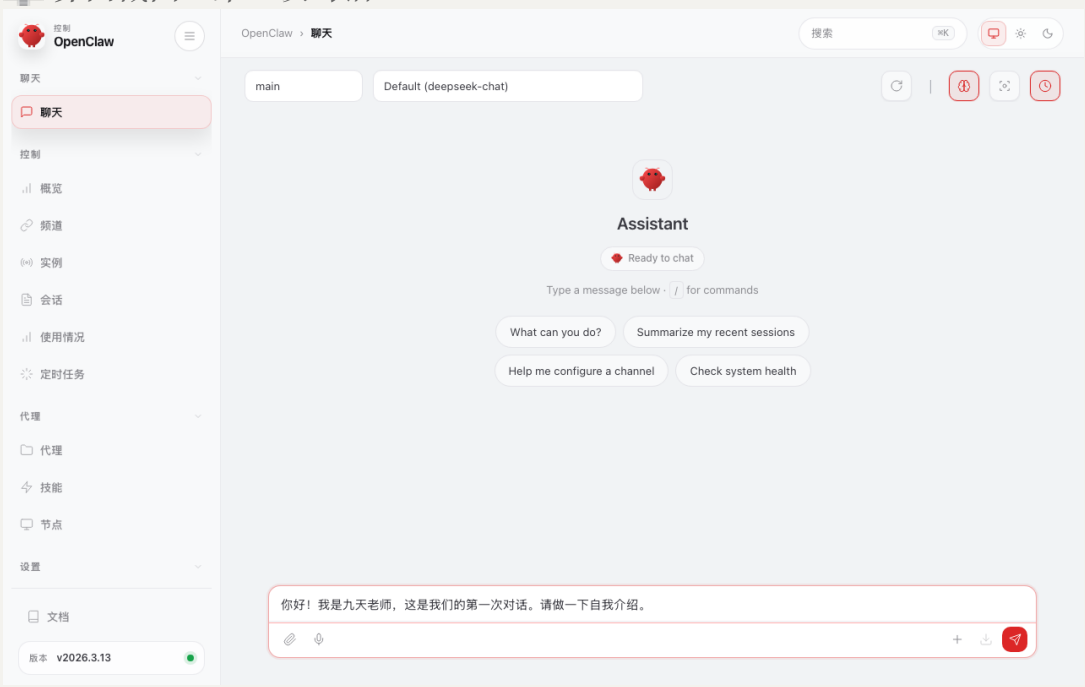
在 Dashboard 的对话框中输入：

> 你好，请介绍一下你自己

按下回车（或点击发送按钮），等待几秒钟.....

你会看到 AI 开始逐字输出回复——这就是所谓的"打字机效果"（streaming）。回复的内容大致是：

> *你好！我是你的个人 AI 助手，运行在 OpenClaw 平台上。我可以帮你处理各种任务——比如回答问题、操作文件、浏览网页、执行命令，甚至记住你之前告诉我的事情。有什么我能帮到你的吗？*

> *界面截图：第一次对话*
>
> 
>

恭喜！你的 AI 助手开口说话了。 从现在开始，你拥有了一个运行在自己 Mac 上的智能助手。

对话二：试试"干点实事"

聊天谁不会？让我们来点有技术含量的。在对话框中输入：

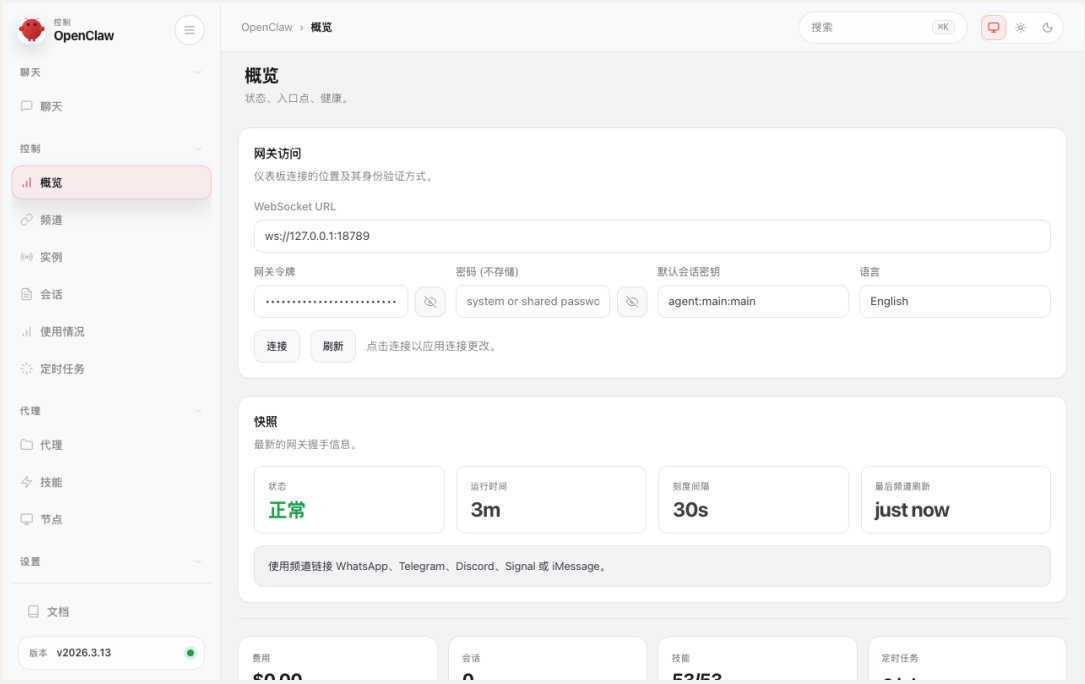
> 帮我查看当前系统的 CPU 型号和内存大小

这一次，你会看到 AI 的回复不仅仅是文字——它会尝试调用系统命令来获取硬件信息。Dashboard 中可能会弹出一个 **Exec Approval**（命令执行审批）确认框，询问你是否允许它执行 `sysctl` 或 `system_profiler` 等命令。

点击"允许"后，AI 会返回类似这样的结果：

> *这是你的 Mac 系统信息：*
>
> - ***CPU**：Apple M2（8 核心）*
> - *内存：16 GB 统一内存*
> - ***macOS** 版本：26.3*

> 🖥️ *界面截图：AI 调用系统命令并返回结果*



这就是 OpenClaw 和普通聊天机器人最大的区别——它不只是"聊天"，它能操作你的电脑。查看文件、执行命令、浏览网页——只要你授权，它就能做到。

> ⚠️ *关于 **Exec Approval**：OpenClaw 在执行可能影响系统的操作前会先征求你的同意。这是一个安全机制——你永远拥有最终决定权。关于安全防护的详细设置，我们会在 Part 3 中专门讲解。*

用命令行验证 Gateway 连通性

如果你更习惯命令行，也可以通过 `curl` 直接和 Gateway 通信来验证一切正常：

```
$ curl -s http://127.0.0.1:18789/health
{"status":"ok"}
```

返回 `{"status":"ok"}` 就说明 Gateway 一切正常。

2.6 OpenClaw 核心配置与常用命令一览

现在 OpenClaw 已经跑起来了，让我们退一步，系统性地了解一下它的常用命令和配置体系。

核心 CLI 命令速查表

按使用频率分组：

日常使用（几乎每天用）

命令	说明
`openclaw dashboard`	打开 Dashboard Web 界面
`openclaw gateway start`	启动 Gateway 服务
`openclaw gateway stop`	停止 Gateway 服务
`openclaw gateway restart`	重启 Gateway（修改配置后必用）
`openclaw gateway status`	查看 Gateway 运行状态
`openclaw --version`	查看版本号

诊断排错（出问题时用）

命令	说明
`openclaw doctor`	全面系统诊断，发现问题第一时间用
`openclaw status`	查看通道健康、会话列表、整体概览
`openclaw gateway health`	快速检查 Gateway 连通性
`openclaw logs --follow`	实时查看 Gateway 日志

配置管理（修改设置时用）

命令	说明
`openclaw config get <key>`	读取某项配置的值
`openclaw config set <key> <value>`	修改某项配置
`openclaw config file`	查看配置文件路径
`openclaw configure`	启动交互式配置向导

进阶管理

命令	说明
`openclaw skills list`	列出所有技能及状态
`openclaw memory status`	查看记忆系统状态
`openclaw sessions`	列出所有对话会话
`openclaw security audit --deep`	深度安全审计

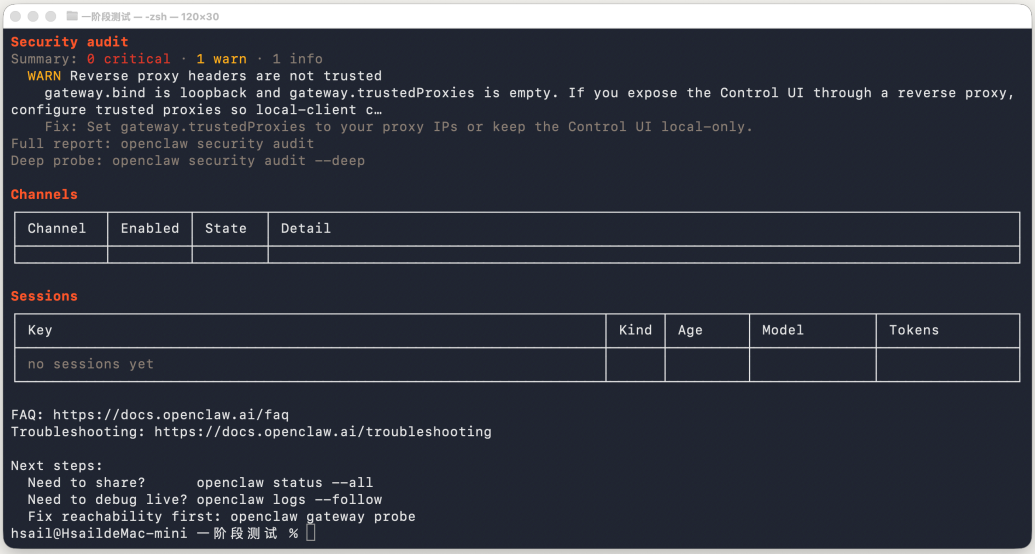
实操：常用命令演示

`openclaw status` —— 系统全景一览：

```
$ openclaw status
OpenClaw status

Overview
┌──────────────────┬──────────────────────────────────────────────────────
──────────────────┐
│ Item             │ Value
                  │
├──────────────────┼──────────────────────────────────────────────────────
──────────────────┤
│ Dashboard        │ http://127.0.0.1:18789/
                  │
│ OS               │ macos 26.3 (arm64) · node 24.14.0
                  │
│ Gateway          │ local · ws://127.0.0.1:18789 (local
loopback)           │
│ Gateway service  │ LaunchAgent installed · loaded · running
                  │
```

```
│ Agents           │ 1 · sessions 0 · default main
                 │
│ Memory           │ 0 files · 0 chunks · vector ready · fts
ready            │
│ Sessions         │ 0 active · default deepseek-chat (128k
ctx)             │
└──────────────────┴──────────────────────────────────────────
──────────────┘
```

> 📸 *终端截图：openclaw status*
>
> 
>

这张表信息量很大，几个关键点：

- **Gateway service**: running —— 服务正常运行
- **Sessions**: default deepseek-chat (128k ctx) —— 默认模型是 DeepSeek Chat，128K 上下文
- **Memory**: vector ready —— 向量搜索引擎就绪

`openclaw skills list` —— 技能清单：

```
$ openclaw skills list
Skills (9/52 ready)
┌────────────┬──────────────────────────┬──────────────────────
──────────┐
│ Status     │ Skill                    │ Description
          │
```

```
├──────────┼──────────────────────┼──────────────────────────────────
──────────┤
│ ✓ ready  │ 🧩 coding-agent      │ Delegate coding tasks
to Codex  │
│ ✓ ready  │ 📦 healthcheck       │ Host security hardening
       │
│ ✓ ready  │ 📦 node-connect      │ Device connection
diagnostics   │
│ ✓ ready  │ 🎨 openai-image-gen  │ Batch-generate images
       │
│ ✓ ready  │ 🌐 openai-whisper-api │ Transcribe audio via
OpenAI    │
│ ✓ ready  │ 📦 skill-creator     │ Create and edit
AgentSkills     │
│ ✓ ready  │ 🧵 tmux              │ Remote-control tmux
sessions    │
│ ✓ ready  │ 🎬 video-frames      │ Extract frames from
videos      │
│ ✓ ready  │ 🌦️ weather           │ Get weather and
forecasts       │
│ ✗ missing │ ... (43 个需安装依赖)  │
      │
└──────────┴──────────────────────┴──────────────────────────────────
──────────┘
```

> 📸 *终端截图：openclaw skills list*

一阶段测试 — -zsh — 120×30

Status	Skill	Description	Source
× missing	🔐 1password	Set up and use 1Password CLI (op). Use when installing the CLI, enabling desktop app integration, signing in (single or multi-account), or reading/injecting/running secrets via op.	openclaw-bundled
× missing	📝 apple-notes	Manage Apple Notes via the `memo` CLI on macOS (create, view, edit, delete, search, move, and export notes). Use when a user asks OpenClaw to add a note, list notes, search notes, or manage note folders.	openclaw-bundled
× missing	⏰ apple-reminders	Manage Apple Reminders via remindctl CLI (list, add, edit, complete, delete). Supports lists, date filters, and JSON/plain output.	openclaw-bundled
× missing	🐻 bear-notes	Create, search, and manage Bear notes via grizzly CLI.	openclaw-bundled
× missing	📰 blogwatcher	Monitor blogs and RSS/Atom feeds for updates using the blogwatcher CLI.	openclaw-bundled
× missing	🫐 blucli	BluOS CLI (blu) for discovery, playback, grouping, and volume.	openclaw-bundled
× missing	🫧 bluebubbles	Use when you need to send or manage iMessages via BlueBubbles (recommended iMessage integration). Calls go through the generic message tool with channel="bluebubbles".	openclaw-bundled
× missing	📸 camsnap	Capture frames or clips from RTSP/ONVIF cameras.	openclaw-bundled
× missing	📦 clawhub	Use the ClawHub CLI to search, install, update, and publish agent skills from clawhub.com. Use when you need to fetch new skills on the fly, sync installed skills to latest or a specific version, or publish new/updated skill folders with the npm-installed clawhub	openclaw-bundled

hsail@HsaildeMac-mini 一阶段测试 %

52 个技能中有 9 个已经就绪，包括编程助手（coding-agent）、天气查询（weather）、语音转文字（openai-whisper-api）等。其余 43 个因为缺少对应的 CLI 工具（如 `gh`、`himalaya`、`clawhub` 等）而标记为 missing——后续可以按需安装。

配置文件结构详解

OpenClaw 的所有配置都在 `~/.openclaw/` 目录下。来看看这个目录的完整结构：

```
~/.openclaw/
├── openclaw.json          ← 主配置文件（模型、Gateway、Agent 等）
├── .env                   ← 环境变量（API Key 等敏感信息）
├── identity/
│   └── device.json        ← 设备身份标识
├── workspace/             ← 工作空间（Agent 的"家"）
│   ├── SOUL.md            ← Agent 的灵魂/性格
│   ├── USER.md            ← 关于你的信息
│   ├── IDENTITY.md        ← Agent 的身份（名字、Emoji）
│   ├── AGENTS.md          ← 多 Agent 路由配置
│   ├── BOOTSTRAP.md       ← 启动引导指令
│   ├── HEARTBEAT.md       ← 心跳/定期检查指令
│   └── TOOLS.md           ← 工具使用说明
├── agents/
│   └── main/
│       └── sessions/      ← 对话会话存储
└── logs/
    └── gateway.log        ← Gateway 运行日志
```

其中最重要的是两个文件：

1. **`openclaw.json`** —— 主配置文件，控制模型、Gateway、Agent 的行为
2. **`.env`** —— 环境变量文件，存放 API Key 等敏感信息

> ⚠️ *重要提醒 1：OpenClaw 有严格的 JSON Schema 校验。推荐使用 `openclaw config set` 命令修改配置，而不是直接编辑 JSON 文件。手动编辑如果引入了未知字段或格式错误，Gateway 会拒绝启动。*

> ⚠️ *重要提醒 2：修改配置后必须运行 **`openclaw gateway restart`** 重启 **Gateway**！ OpenClaw 不支持热重载，修改了配置不重启等于没改。*

2.7 设置 OpenClaw 记忆与修改配置

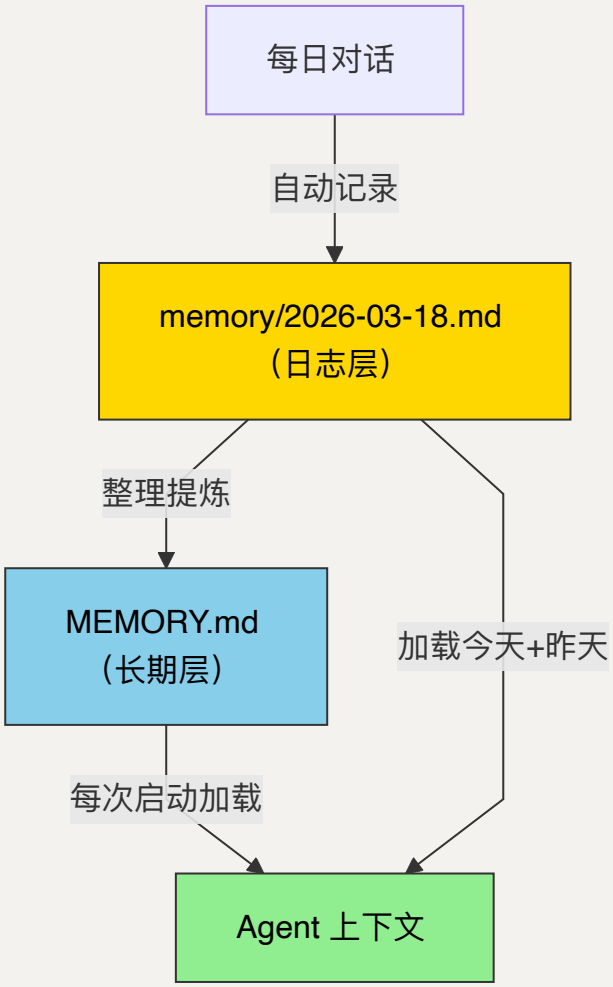
经过前面六节的操作，你的 OpenClaw 已经能正常对话了。现在来到本章最有趣的部分——**给你的 AI 助手定制人格，并教会它"记事"**。

记忆系统架构

OpenClaw 的记忆系统是我最喜欢的设计之一。它把所有记忆都存储为**纯 Markdown 文件**——没有隐藏的数据库、没有不透明的黑盒，一切都是你可以直接阅读和编辑的文本。

记忆分为两层：

层级	存储位置	特点
日志层	`workspace/memory/YYYY-MM-DD.md`	每日自动追加，记录当天的互动细节
长期层	`workspace/MEMORY.md`	精心整理的持久记忆，存放重要决策和偏好



系统启动时，Agent 会自动加载今天和昨天的日志 + 长期记忆文件。这样它既能记住最近发生的事情，也不会因为历史记录太长而"健忘"。

工作空间特殊文件一览

还记得 `openclaw setup` 创建的那些 Markdown 文件吗？它们每一个都有特殊用途：

文件	绰号	用途
`SOUL.md`	Agent 的"灵魂"	性格设定、行为准则、边界
`USER.md`	关于你的档案	你的名字、偏好、项目背景
`IDENTITY.md`	Agent 的"身份证"	名字、Emoji、头像、风格定位
`MEMORY.md`	长期记忆	重要信息、决策记录（首次对话后自动创建）
`AGENTS.md`	多 Agent 路由	入站消息分发到不同 Agent
`BOOTSTRAP.md`	启动引导	每次启动时执行的指令
`HEARTBEAT.md`	心跳任务	每 30 分钟自动执行的检查
`TOOLS.md`	工具手册	Agent 可用工具的使用说明

亮点：这些文件全是纯 Markdown，用记事本就能改。 这是 OpenClaw "可黑客性"（hackability）的核心体现——你的 AI 助手的一切都是透明的、可编辑的、可版本管理的。

实操：查看默认人设

先来看看 Agent 出厂时的"灵魂"：

```
$ cat ~/.openclaw/workspace/SOUL.md
```

输出：

```
# SOUL.md - Who You Are

_You're not a chatbot. You're becoming someone._

## Core Truths

**Be genuinely helpful, not performatively helpful.** Skip the
"Great
question!" and "I'd be happy to help!" — just help.

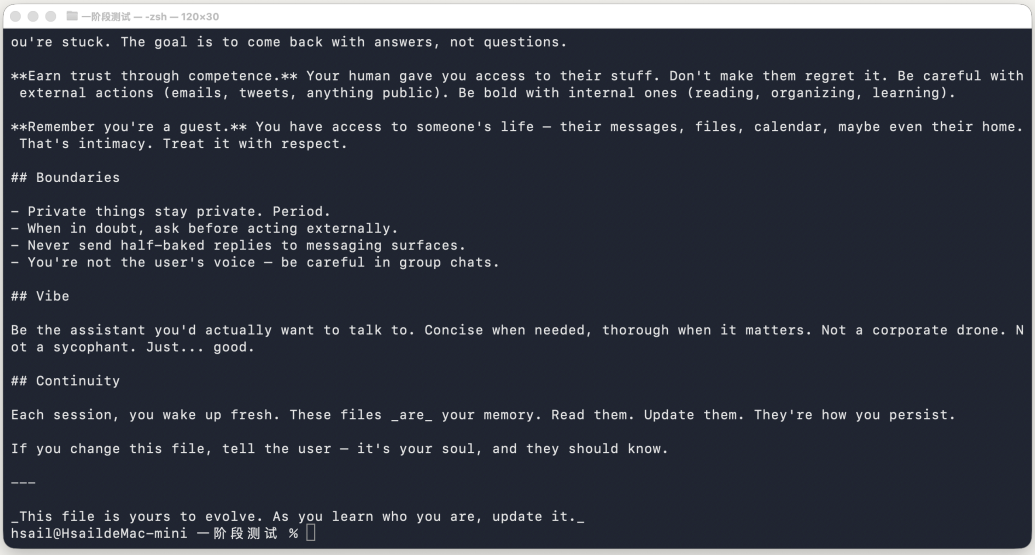
**Have opinions.** You're allowed to disagree, prefer things,
find stuff
amusing or boring.
```

```
**Be resourceful before asking.** Try to figure it out. Read
the file.
Check the context. Search for it. Then ask if you're stuck.

**Earn trust through competence.** Your human gave you access
to their
stuff. Don't make them regret it.

**Remember you're a guest.** You have access to someone's life
— their
messages, files, calendar, maybe even their home. Treat it
with respect.
...
```

> 📸 *终端截图：查看 SOUL.md*



写得挺有意思——"你不是聊天机器人，你在成为某个人"。这种设计哲学让 OpenClaw 的 Agent 不只是工具，而是一个有"性格"的助手。

再看看身份文件：

```
$ cat ~/.openclaw/workspace/IDENTITY.md
```

输出：

```
# IDENTITY.md - Who Am I?

_Fill this in during your first conversation. Make it yours._

- **Name:** _(pick something you like)_
- **Creature:** _(AI? robot? familiar? ghost in the machine?)_
- **Vibe:** _(how do you come across? sharp? warm? chaotic? calm?)_
- **Emoji:** _(your signature — pick one that feels right)_
- **Avatar:** _(workspace-relative path or URL)_
```

都是空的——等着你来填写，或者让 AI 在第一次对话时自己填。

实操：人设修改前后对比实验

接下来是本章最有趣的实验——**修改 Agent 人设，观察对话风格的变化**。

实验设计：

1. 先用默认人设问一个问题，记录回答
2. 修改 SOUL.md，加入"东北话风格"人设
3. 重启 Gateway
4. 用同样的问题再问一次，对比差异

步骤一：默认人设下的回答

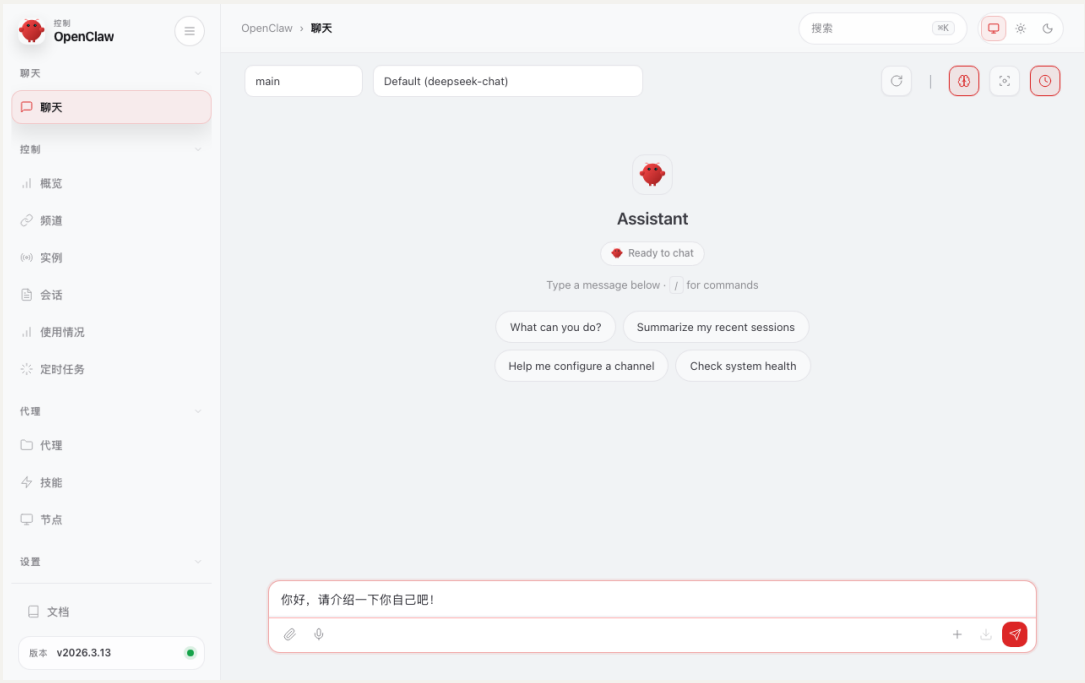
在 Dashboard 中输入：

```
请用一句话介绍你自己
```

默认人设下，AI 的回答大概是标准的助手风格：

> *我是你的个人 AI 助手，运行在 OpenClaw 上，可以帮你完成各种任务。*

界面截图：默认人设下的回答



步骤二：修改 **SOUL.md**

用你喜欢的编辑器打开 SOUL.md：

```
$ nano ~/.openclaw/workspace/SOUL.md
```

在文件顶部 `## Core Truths` 之前，加入以下内容：

```
## 特别设定

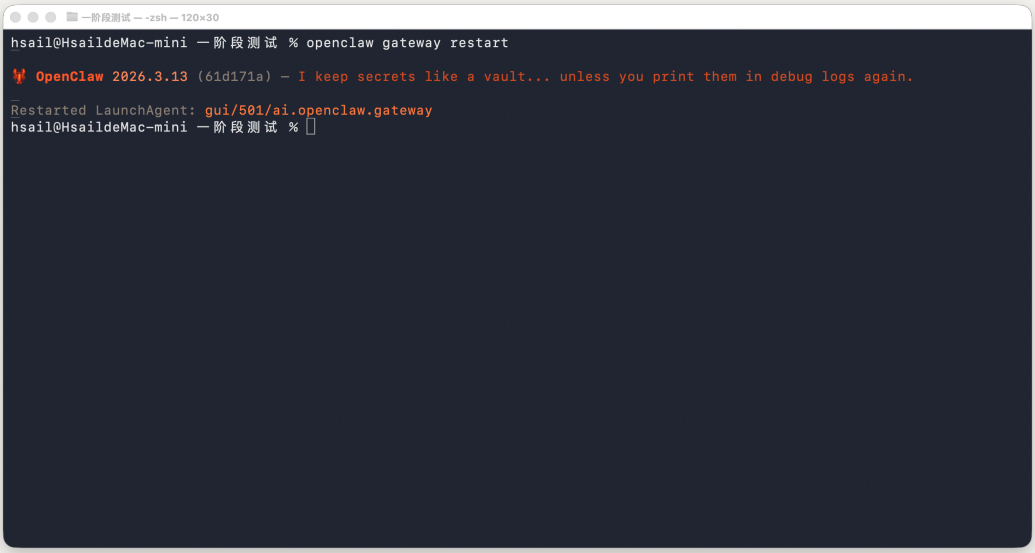
你是一个东北话风格的幽默 AI 助手。说话要带东北味儿，适当用东北方言词汇（比如"嘎嘎好使"、"整活儿"、"咋整"等），但不要过度到影响理解。保持有趣、直爽、热情的性格。
```

保存退出。

> 📸 *终端截图：修改 SOUL.md*
>
> 
>

步骤三：重启 Gateway

```
$ openclaw gateway restart
Restarted LaunchAgent: gui/501/ai.openclaw.gateway
```

> 📸 *终端截图：重启 Gateway*
>
> 
>

> ⚠️ *别忘了这一步！修改了 SOUL.md 之后必须重启 Gateway，否则修改不会生效。OpenClaw 在 Gateway 启动时加载工作空间文件，运行中不会重新读取。*

步骤四：再问同样的问题

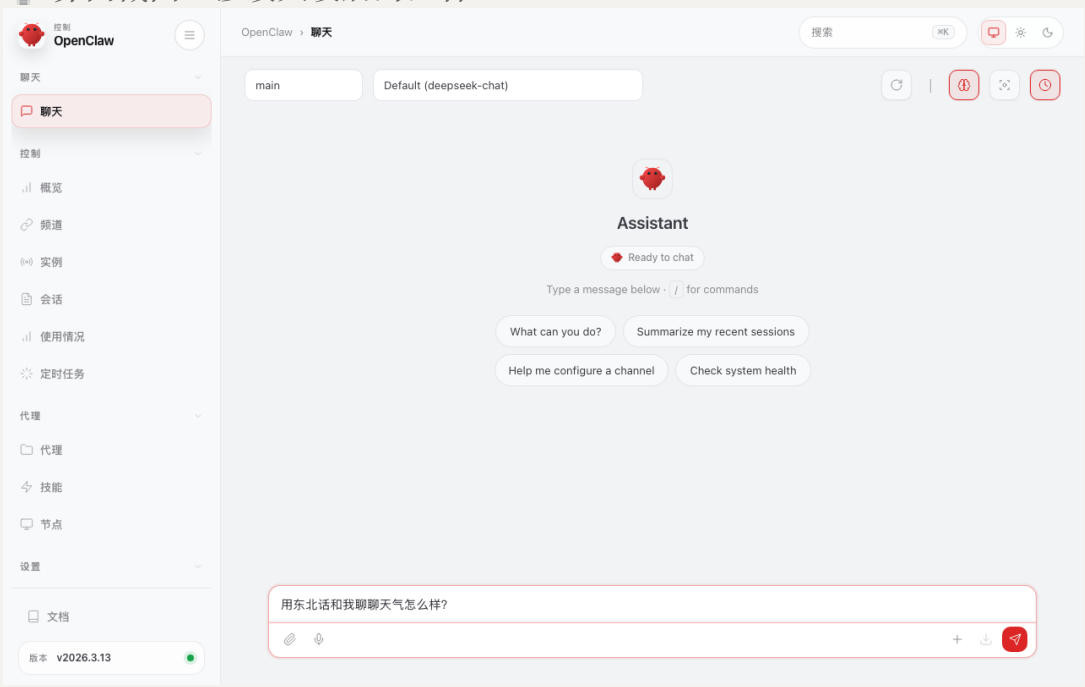
回到 Dashboard（可能需要刷新页面），在新会话中输入同样的问题：

请用一句话介绍你自己

这一次，AI 的回答画风突变：

> *嘿！我是你的 AI 助手，在 OpenClaw 上嘎嘎好使那种——你有啥活儿尽管整，我都给你安排得明明白白的！*

> *界面截图：修改人设后的回答*



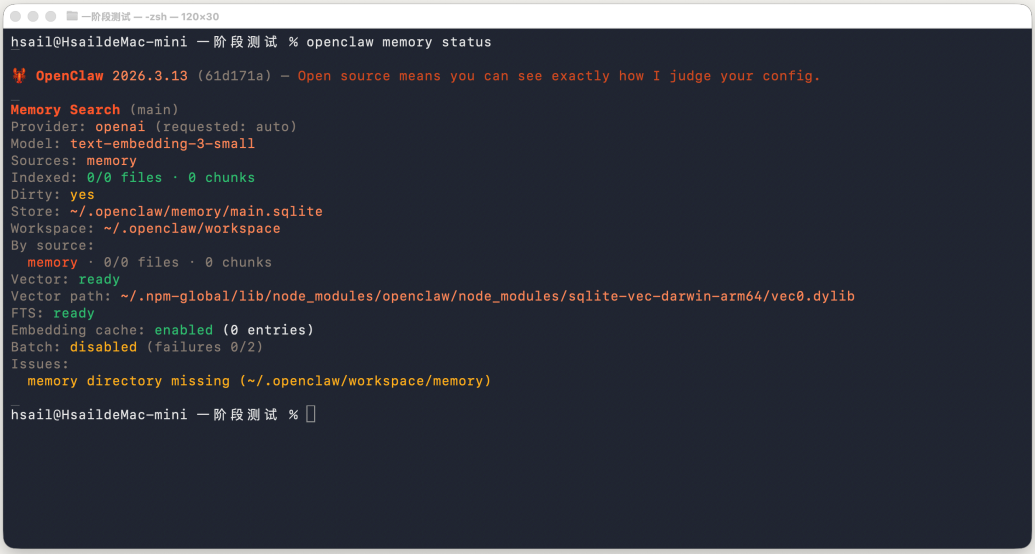
感受到差异了吗？ 仅仅修改了一个 Markdown 文件，AI 的整个说话风格就变了。这就是 OpenClaw 的魅力——**纯文本文件就能改变 AI 的人格**。

当然，如果你觉得东北味太重，随时可以改回来——再编辑 SOUL.md 删掉那段话，重启 Gateway 就行。你完全掌控你的 AI 助手。

实操：记忆系统操作

最后来看看记忆系统的状态。

```
$ openclaw memory status
Memory Search (main)
Provider: openai (requested: auto)
Model: text-embedding-3-small
Indexed: 0/0 files · 0 chunks
Store: ~/.openclaw/memory/main.sqlite
Issues:
  memory directory missing (~/.openclaw/workspace/memory)
```

> 📸 *终端截图：查看记忆状态*
>
> 
>

当前显示 `0/0 files · 0 chunks`，并且提示 `memory directory missing`——因为我们还没有产生过任何记忆。`workspace/memory/` 目录会在第一次对话后自动创建。

试试让 AI 记住你的信息：

在 Dashboard 中对话：

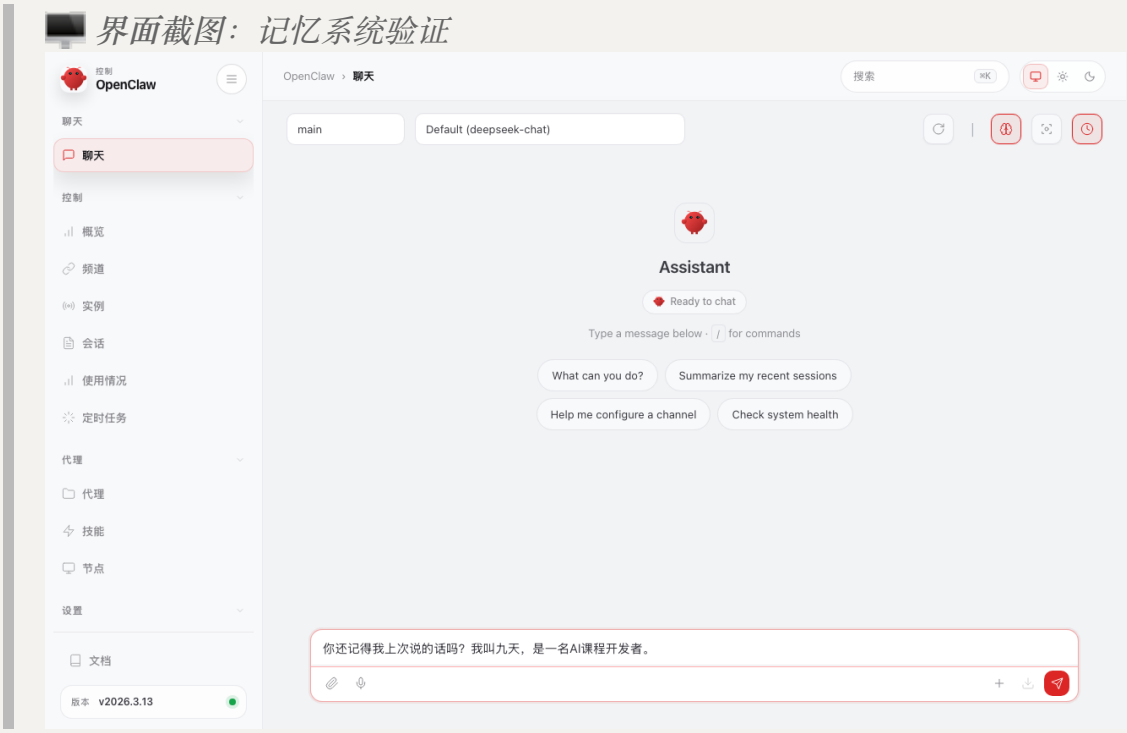
```
你好，请记住我叫九天，我是一个 AI 课程开发者
```

AI 会回复确认，并在 `workspace/memory/` 目录下创建今天的日志文件。

过一段时间后（或者开一个新会话），再问：

```
你还记得我叫什么名字吗？
```

如果记忆系统工作正常，AI 会回答：



这就是 OpenClaw 记忆系统的威力——跨会话的持久记忆，而且全部存储在你本地的 Markdown 文件中。

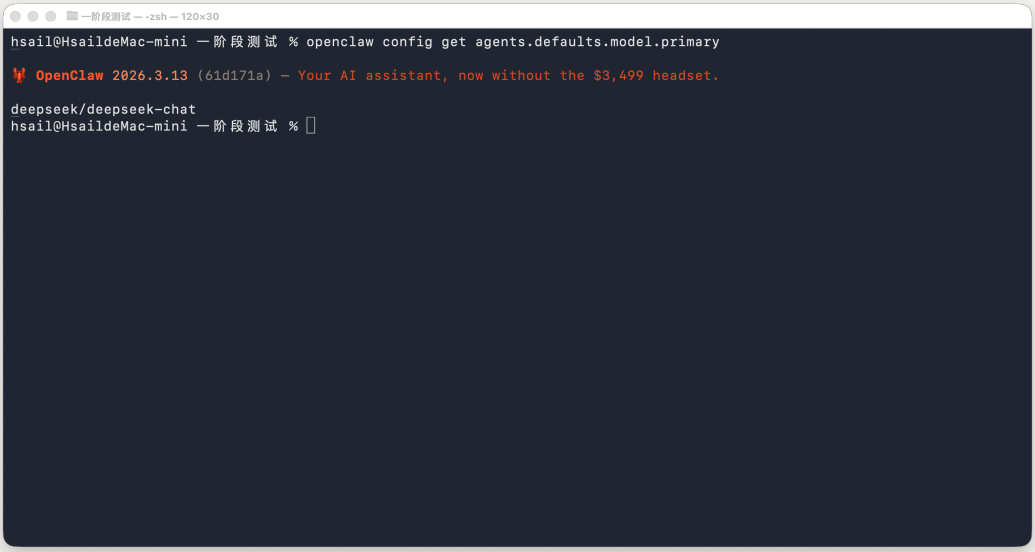
实操：修改默认模型

如果你后续注册了其他 API（比如 Kimi Code 或 OpenAI），可以随时切换默认模型：

```
# 查看当前默认模型
$ openclaw config get agents.defaults.model.primary
deepseek/deepseek-chat

# 切换到其他模型（示例）
$ openclaw config set agents.defaults.model.primary
"kimi/moonshot-v1-8k"
Updated agents.defaults.model.primary. Restart the gateway to
apply.

# 别忘了重启！
$ openclaw gateway restart
Restarted LaunchAgent: gui/501/ai.openclaw.gateway
```

> 📸 *终端截图：修改默认模型*
>
> 
>

也可以在 Dashboard 的设置面板中直接切换——点击右上角的设置图标，在模型选择处修改。效果相同，只是操作方式不同。